

Table of Contents

Step 1: Spring Boot Project creation steps.....	1
Step 2: Understand Spring Boot as opinionated.....	2
Step 3: Understand Object creation in spring boot	4
Step 4: Get bean context	5
Step 5: @Value Annotation.....	5
Step 6: @Autowired and @Service Annotation	6
Step 7: @Qualifier Annotation	8
Step 8: @Bean Annotation.....	10

Step 1: Spring Boot Project creation steps

1. Open below url on the browser:
<https://start.spring.io/>
2. Enter details as shown below:

Project: Maven
Language: java
Spring Boot: Choose latest stable version [NOT Snapshot]
Packaging: jar
Java : 21
Group : com.demo
Artifact: SpringBootDemo

Click on Add Dependencies and add the below 2 dependencies as shown:

Click on generate and the zip will be downloaded. Extract and open the project on intellij.

The screenshot shows the Spring Initializr web form. The following fields and sections are highlighted with red boxes:

- Project:** Maven (selected)
- Language:** Java (selected)
- Spring Boot:** 3.5.4 (selected)
- Project Metadata:**
 - Group: com.demo
 - Artifact: SpringBootDemo
 - Name: SpringBootDemo
 - Description: Demo project for Spring Boot
 - Package name: com.demo.SpringBootDemo
 - Packaging: Jar (selected)
 - Java: 21 (selected)
- Dependencies:** Spring Data JPA and MySQL Driver (both selected)
- Buttons:** ADD DEPENDENCIES... (top right), GENERATE (bottom center), EXPLORE (bottom center)

Step 2: Understand Spring Boot as opinionated

1. View the pom.xml file that has spring boot starter parent.

It is a special starter project that provides default configurations for our application and a complete dependency tree to quickly build our Spring Boot project.

It also provides default configurations for Maven plugins, such as maven-failsafe-plugin, maven-jar-plugin, maven-surefire-plugin, and maven-war-plugin.

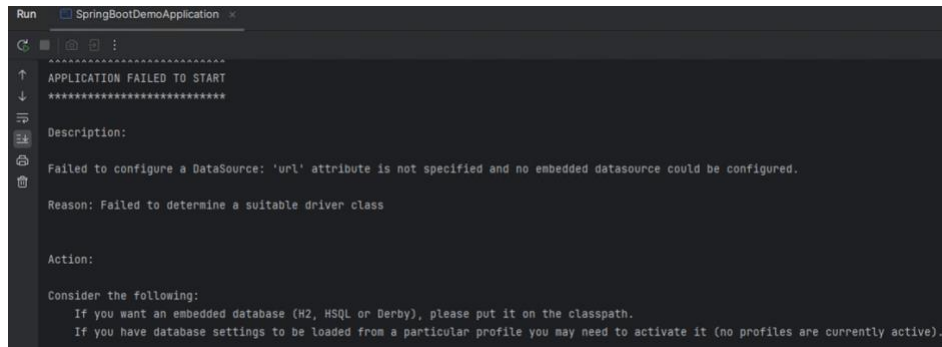
Beyond that, it also inherits dependency management from spring-boot-dependencies, which is the parent to the spring-boot-starter-parent.

2. @SpringBootApplication on the class with the main method:

This annotation is used to enable three features, that is:

- a. @EnableAutoConfiguration: enable Spring Boot's auto-configuration mechanism
- b. @ComponentScan: enable @Component scan on the package where the application is located.

- c. `@Configuration`: allow to register extra beans in the context or import additional configuration classes
3. Run the application, you should see the error with respect to database configuration. This is the default behaviour of spring boot. It sees mysql dependency but did not find database connection parameter information.



4. Open database and create a database/schema named “neueda”
5. Open application.properties file within the resources folder and add the database related connection parameters:

```
spring.datasource.url=jdbc:mysql://localhost:3306/java
spring.datasource.username=root
spring.datasource.password=root1234
spring.datasource.driverClassName=com.mysql.cj.jdbc.Driver
```

```
# below property if used will perform DDL operations to create table
# create will always drop and recreate
# update will create only if none exists
spring.jpa.hibernate.ddl-auto=create
```

```
# below property if used will show the sql queries generated by hibernate
spring.jpa.show-sql=true
```

6. Run the main method again and you should see the error is GONE!!!
7. Following the package structure is very important with spring boot for it to follow the default configurations.

Step 3: Understand Object creation in spring boot

1. Create interface as follows:
package com.demo.SpringBootDemo.component;

```
public interface ITokenGenerator {  
    public String generateToken();  
}
```

2. Create class as follows:

```
package com.demo.SpringBootDemo.component;  
  
import java.util.Random;  
  
public class RandomTokenGenerator implements ITokenGenerator {  
  
    public RandomTokenGenerator () {  
        System.out.println("Token default constructor");  
    }  
    public String generateToken() {  
        return "Token " +new Random().nextInt(1,10);  
    }  
}
```

3. Normally we create object and then call the respective method of the class.
RandomTokenGenerator ob = new RandomTokenGenerator ();
ob.generateToken();
4. Object creation part is taken care by spring. Just add the annotation @Component on the class Token as follows:
The classes whose objects are created by spring are called as “**SPRING MANAGED BEAN**”

```
import org.springframework.stereotype.Component;  
@Component  
public class RandomTokenGenerator implements ITokenGenerator{  
    // other methods  
}
```

5. Just run the main method and you should see the output from the constructor.

```
2024-07-29T16:25:32.870+05:30 INFO 9678 --- [SpringBootDemo] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start compl  
2024-07-29T16:25:32.911+05:30 WARN 9678 --- [SpringBootDemo] [main] org.hibernate.dialect.Dialect : HHH000511: The 5.7.34 vers  
2024-07-29T16:25:33.096+05:30 INFO 9678 --- [SpringBootDemo] [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA platform  
2024-07-29T16:25:33.098+05:30 INFO 9678 --- [SpringBootDemo] [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityMana  
Token default constructor  
2024-07-29T16:25:33.193+05:30 INFO 9678 --- [SpringBootDemo] [main] c.t.S.SpringBootDemoApplication : Started SpringBootDemoAppl  
2024-07-29T16:25:33.196+05:30 INFO 9678 --- [SpringBootDemo] [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerF  
2024-07-29T16:25:33.198+05:30 INFO 9678 --- [SpringBootDemo] [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown in  
2024-07-29T16:25:33.223+05:30 INFO 9678 --- [SpringBootDemo] [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown co
```

Step 4: Get bean context

1. To get access to the bean [class] created by spring, get it from spring context. Update main method as follows:

```
public static void main(String[] args) {

    ApplicationContext context =
        SpringApplication.run(SpringBootDemoApplication.class,
            args);

    RandomTokenGenerator token =
        context.getBean(RandomTokenGenerator.class);
    System.out.println(token.generateToken());

}
```

Step 5: @Value Annotation

1. Instead of manually setting the bounds for generatetoken() function, add origin and bound as instance variables.
2. Add parameterized constructor, getters and setters for the same as follows:
3. DO UPDATE THE generateToken() for the parameters.

```
@Component
public class RandomTokenGenerator {

    private int origin;
    private int bound;
    public RandomTokenGenerator () {
        System.out.println("Token default constructor");
    }

    public int getOrigin() {
        return origin;
    }

    public void setOrigin(int origin) {
        this.origin = origin;
    }

    public int getBound() {
        return bound;
    }

    public void setBound(int bound) {
        this.bound = bound;
    }

    public String generateToken() {
        return "Token " +new Random().nextInt(origin,bound) ;
    }

}
```

4. To set values for origin and bound, spring provides 3 ways to inject values

- a. Field
- b. Constructor
- c. Setter

5. To do field injection update Token class as follows :

```
@Component
public class RandomTokenGenerator {

    @Value("10")
    int origin;
    @Value("20")
    int bound;
    // other methods
}
```

6. Now run the main method and you should get the range of values between 10 and 20.

7. For setter injection, update Token class as follows [CAN CHOOSE TO COMMENT FIELD INJECTION]

```
@Value("20")
public void setOrigin(int origin) {
    this.origin = origin;
}

@Value("30")
public void setBound(int bound) {
    this.bound = bound;
}
```

8. Values can also be passed via the properties file. Add below in application.properties file

```
origin=20
bound=40
```

Step 6: @Autowired and @Service Annotation

1. Consider below class BankService that has a dependency on TokenGenerator class.

```
public class BankService {

    private ITokenGenerator tokenGenerator;

    public BankService() {
        System.out.println("Bank Service default constructor");
    }

    public BankService(ITokenGenerator tokenGenerator) {
        this.tokenGenerator = tokenGenerator;
    }

    public ITokenGenerator getTokenGenerator() {
        return tokenGenerator;
    }

    public void setTokenGenerator(ITokenGenerator
```

```

tokenGenerator) {
    this.tokenGenerator = tokenGenerator;
}
public void getTokenValue() {
    System.out.println(tokenGenerator.generateToken());
}
}

```

2. Use `@Component` annotation on this class for spring to create object of BankService class. Update class as follows:

```

@Component
public class BankService {
}

```

3. Run the main method and will see the output from default constructor of BankService class.
4. Now get access of BankService class in the main method as follows and call the `getTokenValue()` method:

```

BankService bankService = context.getBean(BankService.class);
bankService.getTokenValue();

```

5. It will throw `NullPointerException` for TokenGenerator class.

```

07:05:30 INFO 81044 --- [SpringBootDemo] [ionShutdownHook] j.t.c.a.c.c.a.z.h.e.c.t.y.m.a.s.f.f.a.c.t.o.r.y.d.e.a.n : Closing JPA EntityManagerFactory for 'p
ain" java.lang.NullPointerException Create breakpoint : Cannot invoke "com.training.SpringBootDemo.component.TokenGenerator.generateToken()" bec
ringBootDemo.service.BankService.getTokenValue(BankService.java:28)
ringBootDemo.SpringBootDemoApplication.main(SpringBootDemoApplication.java:26)
5+05:30 INFO 81044 --- [SpringBootDemo] [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown initiated...
2+05:30 INFO 81044 --- [SpringBootDemo] [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown completed.

```

6. Even though TokenGenerator class object was created, spring needs to know it is required by BankService class. Normally we would do as follows:

```

TokenGenerator tokenGenerator = new TokenGenerator();
BankService bankService = new BankService(tokenGenerator);
//OR
bankService.setTokenGenerator(tokenGenerator);

```

7. Hence use `@Autowired` annotation to tell spring to inject the TokenGenerator object created by spring.

It can be done ANY of the following 3 ways.

Use anyone, rerun the program and it should work now.

- a. Field injection: Add `@Autowired` on the field as follows:

```

@Service
public class BankService {
    @Autowired
    private ITokenGenerator tokenGenerator;
    // ...
}

```

- b. Constructor injection:

```

@Autowired
public BankService(ITokenGenerator tokenGenerator) {
    System.out.println("Bank Service parameterized
consturctor");
    this.tokenGenerator = tokenGenerator;
}

```

@Autowired is optional since spring 6 and spring boot 3 if there is only 1 parameterized constructor.

C . Setter injection:

```
@Autowired
public void setTokenGenerator(ITokenGenerator
tokenGenerator) {
    System.out.println("Set token generator");
    this.tokenGenerator = tokenGenerator;
}
```

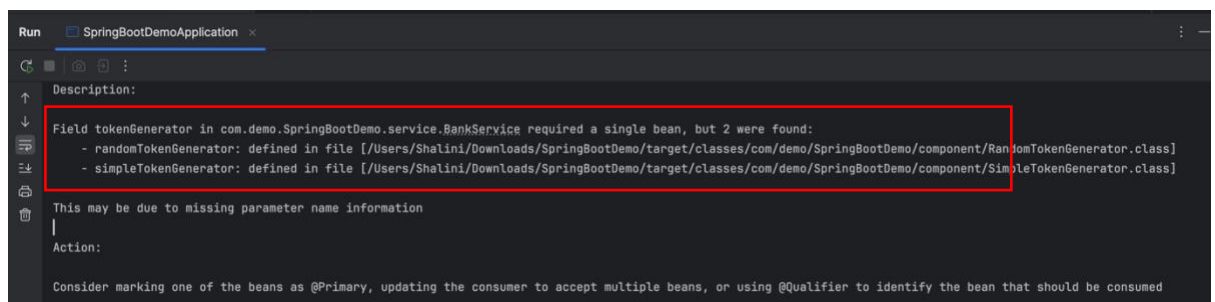
Step 7: @Qualifier Annotation

1. Now suppose we need a simple token generator and not random one. Add below class within component folder:

```
package com.demo.SpringBootDemo.component;
import org.springframework.stereotype.Component;
```

```
@Component
public class SimpleTokenGenerator implements ITokenGenerator{
    private static int counter;
    @Override
    public String generateToken() {
        return "Token " + ++counter;
    }
}
```

2. Running the application gives following error:



3. To resolve this issue, add @Qualifier annotation, mentioning the bean required for injection as follows:
 - a. Field injection:
@Autowired
@Qualifier("simpleTokenGenerator")

```
private ITokenGenerator tokenGenerator;
```

- b. Constructor injection:

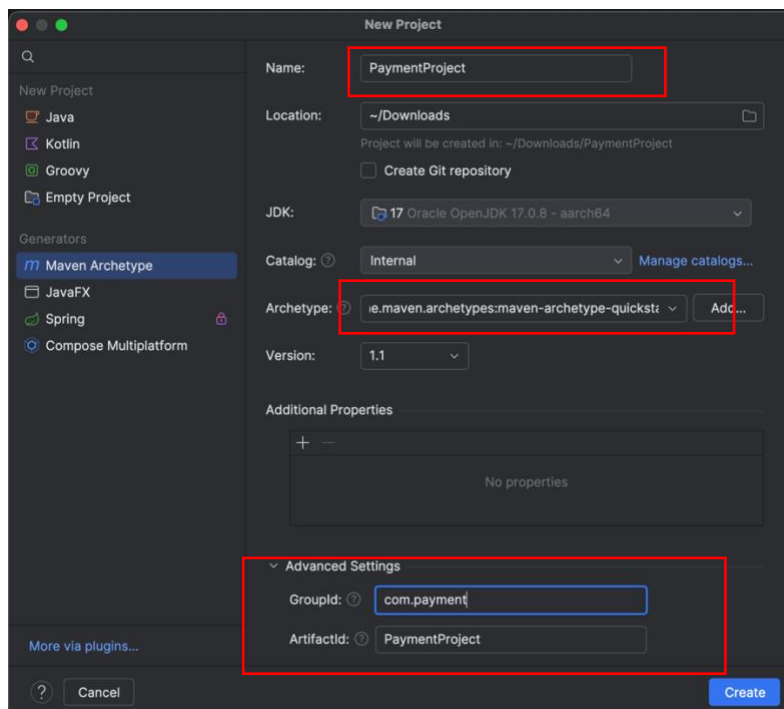
```
public BankService(@Qualifier("simpleTokenGenerator") ITokenGenerator  
tokenGenerator){  
    this.tokenGenerator = tokenGenerator;  
}
```

- c. Setter injection:

```
@Autowired  
@Qualifier("simpleTokenGenerator")  
public void setTokenGenerator(ITokenGenerator){  
    this.tokenGenerator = tokenGenerator;  
}
```

Step 8: @Bean Annotation

1. CREATE a simple maven java project “PaymentProject” with quickstart artifact and create a class as follows:



Below code is within PaymentProject which is a Java Maven Project.
[PLEASE NOTE : IT IS NOT SPRING PROJECT]

It has only one class as follows

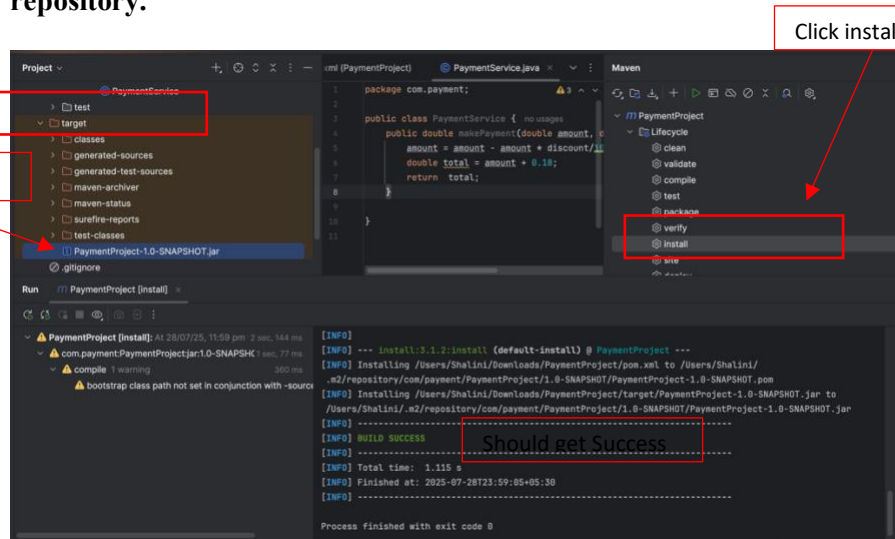
```
package com.payment;
```

```
public class PaymentService {
```

```
    public double makePayment(double amount, double discount){
        amount = amount - amount * discount/100;
        double total = amount + 0.18;
        return total;
    }
}
```

VVIMP:

Run maven install to create a jar file of PaymentProject and install within local repository.



2. Add PaymentProject as maven dependency in pom.xml of **SpringBootDemo** project

```
<dependency>
  <groupId>com.payment</groupId>
  <artifactId>PaymentProject</artifactId>
  <version>1.0-SNAPSHOT</version>
</dependency>
```

3. Add PaymentService as a dependency in BillingService of **SpringBootDemo** project as follows

@Component

```
public class BillingService {
```

```
/**
```

```
 * payment service is not a part of this project and has been added as a dependency.
```

```
 * IN this case we cannot add @Component on the PaymentService class.
```

```
*/
```

```
@Autowired
```

```
private PaymentService paymentService;
```

```
public void calculateCustomerPayment ()
```

```
{
```

```
    System.out.println(paymentService.makePayment(12000,10));
```

```
}
```

```
}
```

- 4 . To inject PaymentService as a dependency, use @Bean annotation as follows within the AppConfig class :

```
package com.demo.SpringBootDemo.config;

@Configuration
public class AppConfig
{
    @Bean
    public PaymentService service() {
        return new PaymentService();
    }
}
```

5. Update the App class main method as follows:

```
BillingService bservice = context.getBean(BillingService.class);
bservice.calculateCustomerPayment();
```

6. Can also get reference of beans using CommandLineRunner as follows:

```
@Bean
public CommandLineRunner commandLineRunner(BillingService billingService) {
    return args -> {
        // Use the Billing Service component
        System.out.println("Amount Calculated");
        billingService.calculateCustomerPayment();
        System.out.println();
    };
}
```